💳 Project: Credit Card Data Pipeline

Architecting for Failure: A Metadata-Driven Lakehouse Implementation

This project implements a robust, incremental data engineering pipeline using a Medallion Architecture. It processes credit card transaction data from a raw state to a highly optimized Star Schema in a Gold Layer, ready for BI reporting and Fraud Detection analysis.

🗺️ Phase 1: The Incremental Ingestion Framework

Objective: Replace "Full Loads" with a state-aware pipeline to minimize compute costs and ensure restartability.

1. **Watermark Table Setup:** Created a control table WatermarkSetting in Azure SQL Database to store the Last_Processed_Date.
2. **Lookup Activity:** Configured an ADF Lookup to fetch the current watermark value.
3. **Source Filtering:** Used a dynamic SQL query in the ADF Copy Activity:
    - SELECT * FROM Transactions WHERE TransactionDateTime > '@{item().LastDate}'
4. **Stored Procedure (The Closer):** Upon a successful copy to the **Bronze Layer** (ADLS Gen2), a Stored Procedure updates the WatermarkSetting table with the MAX(TransactionDateTime) from the source.
    - *Resilience:* If the copy fails, the watermark is never updated, preventing data gaps.

🏛️ Phase 2: Platform Infrastructure & Governance

Objective: Establish a production-grade environment using **Unity Catalog** and **Identity-Based Security**.

1. **Workspace Provisioning:** Deployed a Databricks Workspace attached to a Unity Catalog Metastore.
2. **Access Connector for Azure Databricks:** * Created a **System-Assigned Managed Identity** via the Access Connector.
    - Granted this identity **Storage Blob Data Contributor** permissions on the ADLS Gen2 account.
3. **Unity Catalog Integration:** * **Storage Credential:** Created a UC credential using the Access Connector's Resource ID.
    - **External Locations:** Defined three external locations mapping to the bronze, silver, and gold containers. This ensures Databricks "knows" exactly where the physical data lives without needing Access Keys.
4. **Security Best Practice:** No data is stored in the local DBFS. All tables are **External Tables**, ensuring storage and compute remain decoupled.

🛠️ Phase 3: Medallion Transformations (The Spark Engine)

Objective: Transform raw ingestion into a query-optimized Star Schema.

Step 1: Bronze to Silver (Hardening)

- Read raw Parquet files from the Bronze external location.
- **Schema Enforcement:** Cast transactionAmount to Decimal(18,2) and isFraud to Boolean.
- **Deduplication:** Performed a dropDuplicates() based on the transaction fingerprint to ensure data quality.

Step 2: Silver to Gold Dimensions (Surrogate Keys)

Following your handwritten notes, the dimensions were updated incrementally:

- **Left Join Logic:** Join the incoming Silver data with the existing Gold dimension table.
- **Identification:** Filtered for records where the Gold surrogate_key was NULL.
- **Key Assignment:** Generated new keys using:
  $$NewKey = row_number() + max(ExistingKey)$$

Step 3: Silver to Gold Fact (The Upsert)

- **Joining Keys:** Joined the transaction records with the Account, Merchant, and Card dimensions to replace natural keys with surrogate keys.
- **Delta Merge:** * Created a txn_hash (SHA-256) for every transaction.
  - Executed a **Delta Merge** (Upsert) using the hash.
  - *Result:* If a record exists, it updates; if not, it inserts. This makes the Fact table **Idempotent**.

⚡ Phase 4: Performance & Production Optimization

Objective: Solving the "Small File Problem" and optimizing query latency.

1. **Daily Partitioning:** Partitioned the Fact table by transactionDate (not timestamp) to limit the scan range for the Delta engine.
2. **Repartitioning:** Used .repartition("transactionDate") before the write operation to ensure Spark tasks were aligned with ADLS folder structures.
3. **Auto-Merge:** Enabled spark.databricks.delta.schema.autoMerge.enabled to allow the Gold schema to evolve safely as new features are added.
4. **Z-Ordering:** Ran OPTIMIZE commands on the txn_hash column to accelerate lookups during the Merge phase.

📊 Summary of Major Tool Roles

- **ADF:** Orchestration and state-based movement.
- **Azure SQL:** Metadata and Watermark storage.
- **ADLS Gen2:** Distributed storage with folder-level IAM.
- **Access Connector:** Managed Identity bridge between Databricks and Azure.
- **Unity Catalog:** Governance, Data Lineage, and External Location management.
- **Delta Lake:** ACID compliance and Upsert capability.